

ArchiHotel — Reservation Handling & Check-In

Team C2F1G4 · Katrin Schwab · Soyeong Jeong · Fiona Aurelia

Technical University of Munich — School of Computation, Information and Technology

Information Systems · EAM and Reference Models — Case Study Final Presentation

Heilbronn · 06.07.2026

Introduction: The ArchiHotel Case

Luxury that feels effortless — because the systems behind it are already in sync

ArchiHotel — a luxury hotel group: “where luxury feels effortless”

People, systems and spaces are aligned before the guest ever notices

Every department works from the same shared data:

- Reception → Reservations, Open Costs
- Housekeeping → Arrival Schedule, room status
- Concierge → Transfers, VIPs, Special Requests
- Operational leads → a complete view of the day

Everything converges on the Hotel Management System (HMS)

Our focus area — F1: Reservation Handling & Check-In

The Abstract EA Model

Whole architecture · all three layers · 70 elements / 95 relationships

The whole architecture at an abstract level

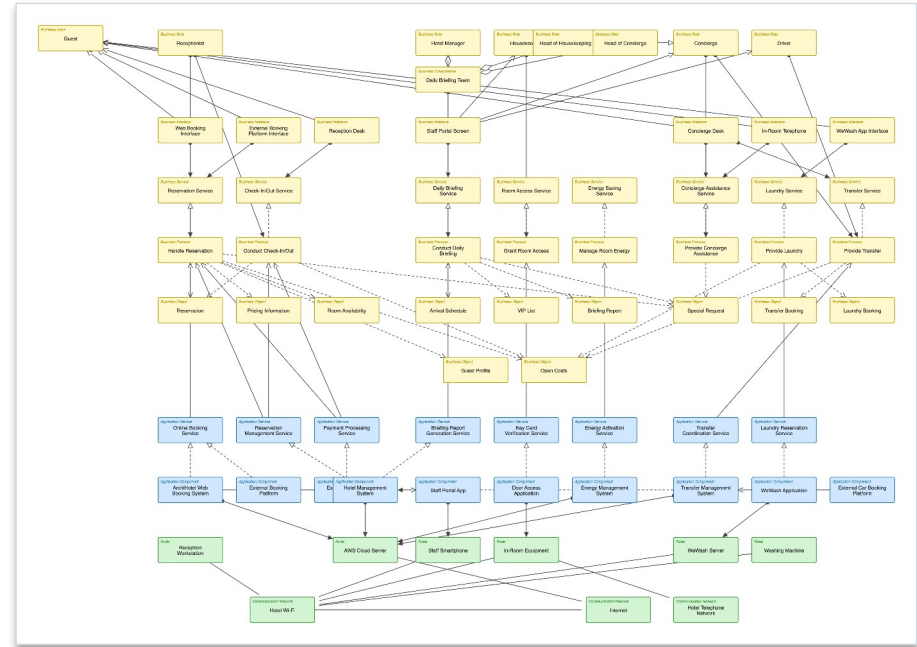
- An overview — don't read every box

HMS sits at the structural centre

AWS Cloud Server hosts the core systems

Three networks: Wi-Fi, Internet, Telephone

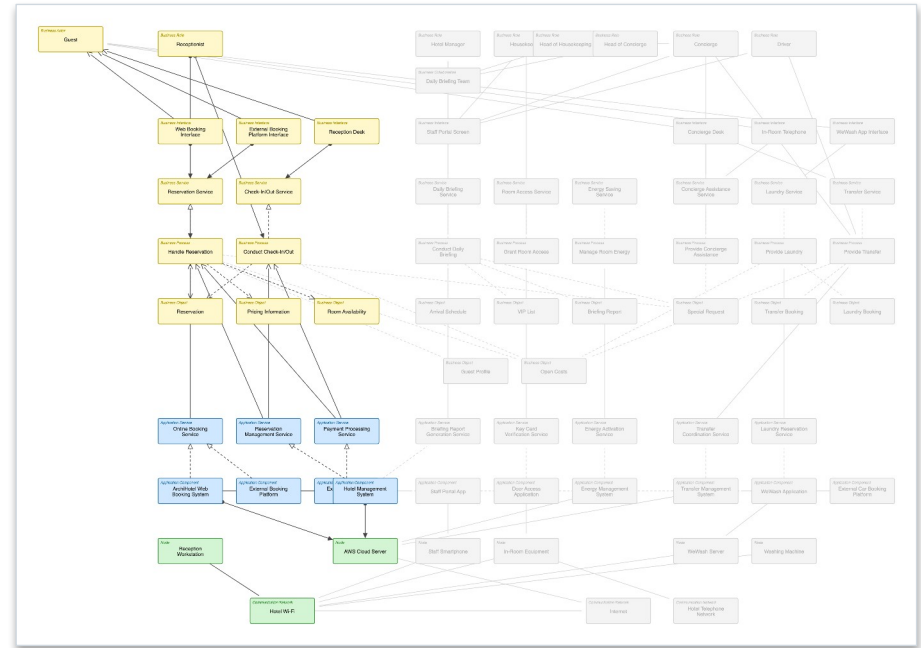
→ We zoom into the F1 region next



Focus Area F1: From Abstract to Detailed

Highlight the F1 region, then zoom in — layer by layer

F1 = the guest's pre-arrival → arrival journey
Booking — website or external platform
Availability & pricing → reserve → payment
Special requests → arrival schedule → briefing
Check-in → issue & configure key card
→ Now layer by layer



Detailed Model — Business Layer

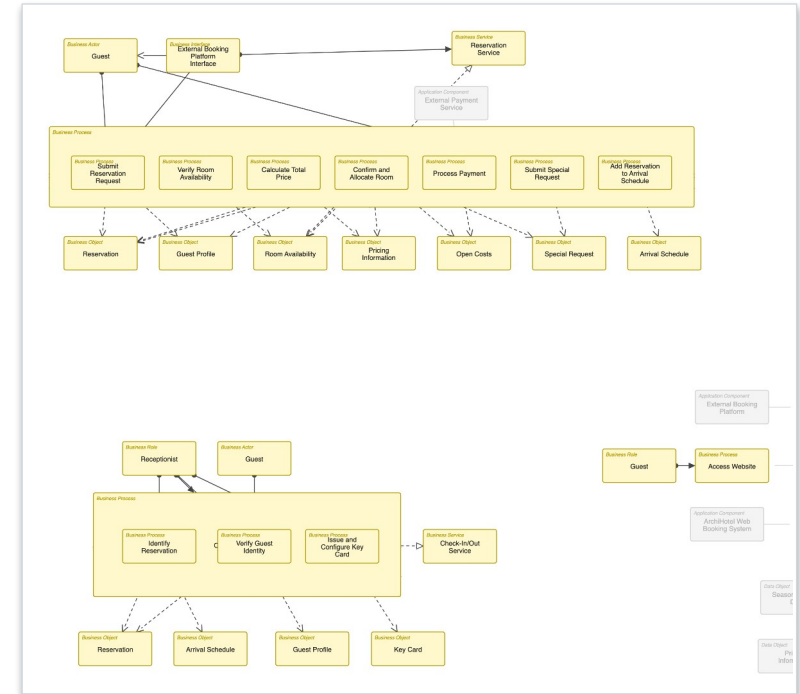
Concrete process steps, roles and business objects

Roles: Guest, Receptionist, Hotel Manager, Heads of Concierge & Housekeeping

Concrete steps (not one abstract process):

- request → availability → price → reserve
- payment (external) → special requests
- arrival schedule → briefing → check-in → key card

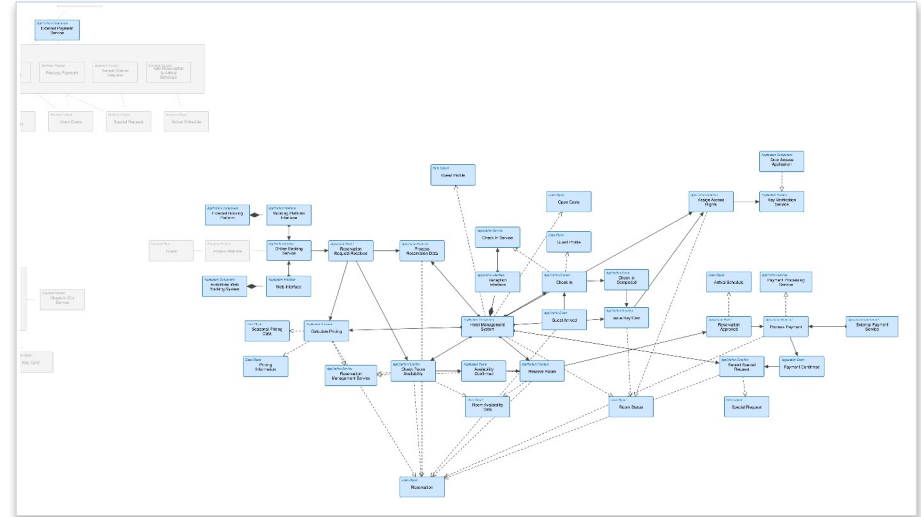
Objects: Reservation, Room Availability, Pricing, Open Costs, Special Request, Arrival Schedule, Briefing Report, Key Card



Detailed Model — Application Layer

Internal structure of the applications — hidden in the abstract view

HMS is the central application component
Internal functions: availability check, pricing, reservation mgmt, payment handling, briefing-report generation, check-in / key-card mgmt
Around it: Web Booking System, External Booking Platform, External Payment Service, Door Access App
Data objects + explicit interfaces (Web, Booking-Platform, Payment-Gateway)



Detailed Model — Technology Layer

Nodes, networks and communication — modelled explicitly

Nodes: AWS Cloud Server; Reception Workstation (PC + phone + payment terminal); Staff Android smartphones; Key-Card Encoder

Networks (explicit): Hotel Wi-Fi (reception & Staff Portal sync); Internet (external booking, payment, AWS)

Communication paths: Reception ↔ HMS (Wi-Fi); HMS ↔ External Payment (Internet); staff phones ↔ HMS (Wi-Fi)

To be built in Archi

Layer not yet modelled. Add the Nodes, Networks and communication paths above, then re-run `render_archi.py` to drop the real diagram here.

Value

- HMS is the visible convergence point
- Change becomes concrete — “what breaks if we swap the payment provider?” = one re-bound link
- Names a shared vocabulary of core objects

Limitations

- An as-is snapshot — no governance
- Lossy abstractions (devices collapsed; Association used for Path)
- Material lacks a motivation layer, SLAs for the AWS single point of failure, and data ownership

Challenges

- Behaviour-element rules (L06) taught after we modelled
- Repeated level-of-abstraction trade-offs
- Archi tooling quirks + a convention fix (Composition → Assignment)

Summary

Thank you — questions?

Reservation & Check-In across all three ArchiMate layers, converging on the HMS
Abstract whole-EA view → detailed F1 view (concrete steps, application structure, explicit communication)
Value: shared vocabulary + change visibility
Limitation: as-is, no motivation layer
Takeaway: EAM is as much teamwork & abstraction judgment as notation

